Low Level Design

Finance Quiz /Tutor Bot

Course Name: Gen AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Mahima Patidar	EN22CS301569
2	Moin Husain	EN22CS301612
3	Nidhish Mehta	EN22CS301642
4	Nikhil Rathore	EN22CS301650
5	Nitisha Kushwah	EN22CS301664

Group Name:12D6

Project Number:GAI-23

Industry Mentor Name:

University Mentor Name: Prof.Ajaj Khan

Academic Year:2026

1.Introduction

1.1 Scope of the Project

The **Finance Quiz / Tutor Bot System** is designed to provide an interactive learning platform that combines quiz-based assessment with an intelligent tutoring system. The application aims to enhance users' financial knowledge through structured quizzes and real-time explanations.

Functional Scope

- User registration and authentication
- Topic-based quiz generation (e.g., Banking, Investment, Taxation)
- Difficulty-level selection (Easy, Medium, Hard)
- Real-time quiz evaluation
- AI-based tutor for answering finance-related queries
- Performance tracking and analytics
- Admin panel for managing questions and users

Non-Functional Scope

- High system performance (low latency responses)
- Secure data storage and communication
- Scalable architecture for multiple concurrent users
- High availability and reliability

1.2 Intended Audience

The system is intended for:

- Students pursuing finance-related studies
- Competitive exam aspirants (Banking, MBA, UPSC)
- Beginners in financial literacy
- Professionals seeking to improve finance knowledge

1.3 System Overview

The system follows a **client-server architecture** where the frontend interacts with backend services through APIs.

Main Components

- **Frontend Layer:** User interface (Web/Mobile)
- **Backend Layer:** Business logic and processing
- **Database Layer:** Data storage
- **AI Tutor Engine:** Handles intelligent responses

Overview Diagram

```
User → Frontend → Backend → Database
              ↓
            AI Tutor
```

2. System Design

2.1 Application Design

The system uses a **3-tier architecture**:

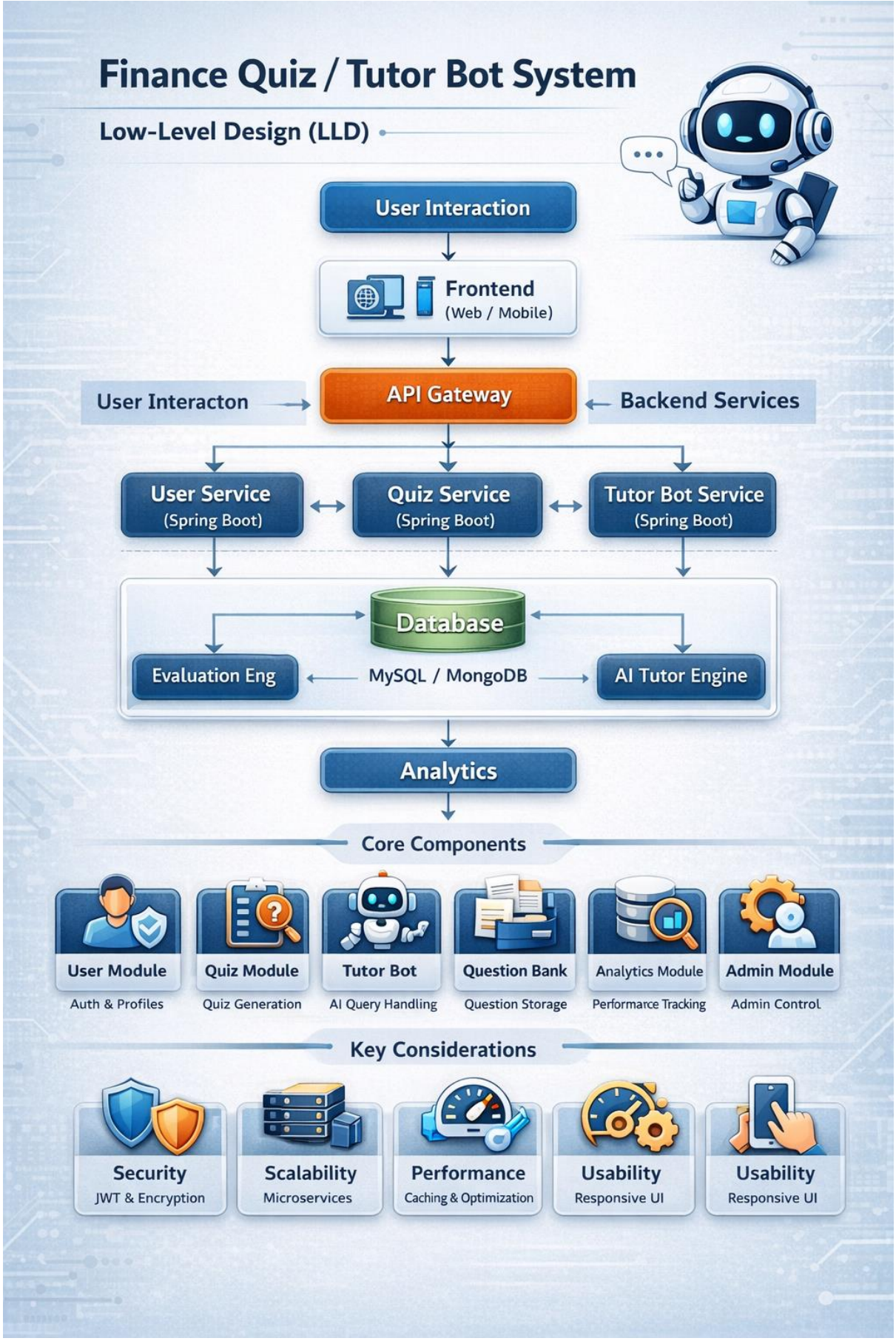
1. **Presentation Layer** – Handles UI/UX
2. **Business Logic Layer** – Processes application logic
3. **Data Layer** – Manages database operations

Design Patterns Used

- **MVC (Model-View-Controller):** Separates logic and UI
- **DAO (Data Access Object):** Handles database interaction
- **Factory Pattern:** Generates quiz dynamically
- **Singleton Pattern:** Manages shared resources

Technology Stack

- Frontend: HTML, CSS, JavaScript / React
- Backend: Java (Spring Boot)
- Database: MySQL / MongoDB
- Authentication: JWT


Finance Quiz / Tutor Bot System
Low-Level Design (LLD)
User Interaction
Frontend
(Web / Mobile)
User Interacton
API Gateway
Backend Services
User Service
(Spring Boot)
Quiz Service
(Spring Boot)
Tutor Bot Service
(Spring Boot)
Database
Evaluation Eng
MySQL / MongoDB
AI Tutor Engine
Analytics
Core Components
User Module
Auth & Profiles
Quiz Module
Quiz Generation
Tutor Bot
AI Query Handling
Question Bank
Question Storage
Analytics Module
Performance Tracking
Admin Module
Admin Control
Key Considerations
Security
JWT & Encryption
Scalability
Microservices
Performance
Caching & Optimization
Usability
Responsive UI
Usability
Responsive UI

2.2 Process Flow

Process Flow Description

1. User logs in or registers
2. Authentication is validated
3. User selects mode (Quiz or Tutor)
4. System processes request based on selection
5. Results or responses are generated
6. Data is stored for analytics

Process Flow Diagram

```
[Start]
  ↓
[Login/Register]
  ↓
[Authentication]
  ↓
[Select Mode]
  ↓
┌──────────────────────┬──────────────────────┐
▼                      ▼
[Quiz Mode]   [Tutor Mode]
  │                    │
  ▼                    ▼
[Load Questions] [User Query]
  │                    │
  ▼                    ▼
[Submit Answers] [Process Query]
  │                    │
  ▼                    ▼
[Evaluate Score] [Generate Response]
  │                    │
  └──────────┬─────────┘
             ▼
        [Store Data]
             ↓
        [Analytics]
             ↓
          [End]
```

2.3 Application Flow

Quiz Flow

1. User selects topic and difficulty
2. System retrieves questions from database
3. Questions are randomized
4. User submits answers
5. Evaluation engine calculates score
6. Results and explanations are displayed
7. Attempt is stored in database

Tutor Flow

1. User enters a finance-related query
2. System processes query using NLP
3. Relevant information is fetched
4. AI generates response
5. Suggestions for further learning are provided

2.4 Component Design

Major Components

1. User Module

- Handles authentication and authorization
- Manages user profiles

2. Quiz Module

- Generates quizzes dynamically
- Handles question selection and randomization

3. Evaluation Engine

- Compares user answers with correct answers
- Calculates score and generates feedback

4. Tutor Bot Module

- Processes natural language queries
- Generates explanations and suggestions

5. Question Bank

- Stores all quiz questions
- Supports CRUD operations

6. Analytics Module

- Tracks user performance
- Identifies weak areas

7. Admin Module

- Manages questions and users
- Monitors system

Component Interaction Diagram

```
[Frontend]
   |
   ▼
[API Gateway]
   |
   ┌──┴──────────────┐
   ▼          ▼
[User Service]  [Quiz Service]
   |          |
   ▼          ▼
[Database]   [Evaluation Engine]
             |
             ▼
          [Tutor Bot]
             |
             ▼
          [Analytics]
```

2.5 Key Design Considerations

Security

- JWT-based authentication
- Password encryption (BCrypt)
- Secure APIs using HTTPS

Performance

- Caching using Redis
- Optimized database queries
- Efficient data retrieval

Scalability

- Modular architecture
- Microservices-ready
- Load balancing support

Reliability

- Error handling mechanisms
- Data consistency
- Backup strategies

Usability

- Simple and intuitive UI
- Responsive design
- Real-time feedback

Maintainability

- Clean code practices
- Layered architect

2.6 API Catalogue

Authentication APIs

- POST /api/auth/register – Register user
- POST /api/auth/login – Login user

Quiz APIs

- GET /api/quiz/start?topic={topic} – Start quiz
- POST /api/quiz/submit – Submit answers
- GET /api/quiz/result/{userId} – Get quiz result

Tutor APIs

- POST /api/tutor/ask – Ask a question
- GET /api/tutor/topics – Get available topics

User APIs

- GET /api/user/profile – Get user profile
- GET /api/user/progress – Get performance data

Admin APIs

- POST /api/admin/question – Add question
- PUT /api/admin/question/{id} – Update question
- DELETE /api/admin/question/{id} – Delete question

3.Conclusion

The **Finance Quiz / Tutor Bot System** is designed with a robust low-level architecture that ensures:

- Efficient quiz generation and evaluation
- Intelligent tutoring capabilities
- Scalable and secure system design

This LLD provides a clear blueprint for implementation and future enhancements.

References

1. Oracle Java EE Documentation
2. Spring Boot Official Documentation
3. *Clean Architecture* – Robert C. Martin
4. *Designing Data-Intensive Applications* – Martin Kleppmann
5. REST API Design Best Practices
6. Database System Concepts – Korth